Context Is All You Need：快手后端AI Coding的实践与思考

快手技术 快手技术 2026年1月29日 18:02 北京

导读

过去数月，快手内部围绕真实需求场景，探索 Java 后端 AI Coding 落地方案。经探索，快手主站团队将真实需求的 AI 代码采纳率逐步提升至60%~90%，沉淀出面向后端的“码图”框架，初步摸索出人与 AI 协同的 AI Native 研发范式。本文将结合真实案例，深入分析 Java 后端 AI Coding 的难点及解决方案。

一、从一个真实案例说起

在实际项目开发中，我们曾碰到这样一个真实需求：要在某个存量后台页面中，新增一个筛选条件，同时新增若干字段用于列表展示。从需求描述来看，这属于非常典型且复杂度不高的改动。

然而，当我们真正着手拆解这个需求时，就会发现它并非表面那么简单，而是横跨了完整的服务端调用链。具体涵盖以下几个关键环节：

- Controller 层负责参数的封装与校验；
- Service 层进行多服务的协同查询；
- DAO 层执行列表查询以及总数统计；
- VO / Helper 层完成数据的组装与结构映射；
- DB 层则要进行字段评估与结构调整。

最终，仅仅是这两个看似微小的改动点，实际涉及的工作量却相当可观，包括 7个类的调整、8 处关键代码的修改，还需要确保多层隐式依赖保持一致。

改动流程示意

详细调用链路图

1、把业务需求直接“扔给 AI”会发生什么？

在没有任何额外上下文和缺少工具辅助的情况下，我们尝试直接将需求描述交给 AI，让它“帮忙改代码”。结果并不意外，且问题非常典型。

问题一：漏改——只看到函数，看不到流程

在该接口中，列表查询和总数统计是紧密关联、成对出现的逻辑，分别负责分页数据和分页总数的处理。但 AI 修改代码时，只对列表查询条件进行了调整，却遗漏了总数统计逻辑，导致前端分页信息与实际数据严重不符。这表明，AI 虽然能处理局部逻辑，但无法把握整体流程的正确性。

问题二：错改——不理解数据结构与职责边界

新增的展示字段来自多个 Service，需经 Helper 统一组装后返回。然而，AI 并不清楚哪些字段已经存在于 Service 返回中，也不了解字段应落在哪一层 VO 结构中。实际编写结果要么留下“TODO”或默认值，要么将字段错误写入顶层对象，破坏了原有的数据结构。由此可见，AI 虽能“写代码”，却并不真正“理解代码在系统中的位置”。

上述这些问题分布在 Controller、Service、DAO、VO各层，但究其本质，原因是一样的：AI只能感知局部上下文，无法感知完整的调用链以及隐式依赖。它能正确修改一个函数，却无法判断这个函数是否与其他逻辑成对出现、修改是否需要在调用链上同步传播，以及数据在不同层级中的真实语义和归属。

二、后端AI Coding难在哪里？

在行业内，前端 AI Coding 领先于后端已成为普遍共识。那么，为何后端 AI Coding（尤其是 Java 端）的推进速度相对迟缓呢？在不断探索的过程中，我们逐渐认识到：AI Coding 理解前端工程，如同“搭积木”般直观；而面对后端工程，则仿佛置身于“走迷宫”的困境。如何让 AI 顺利穿越这座“迷宫”，正是后端 AI Coding 需要攻克的核心难题。

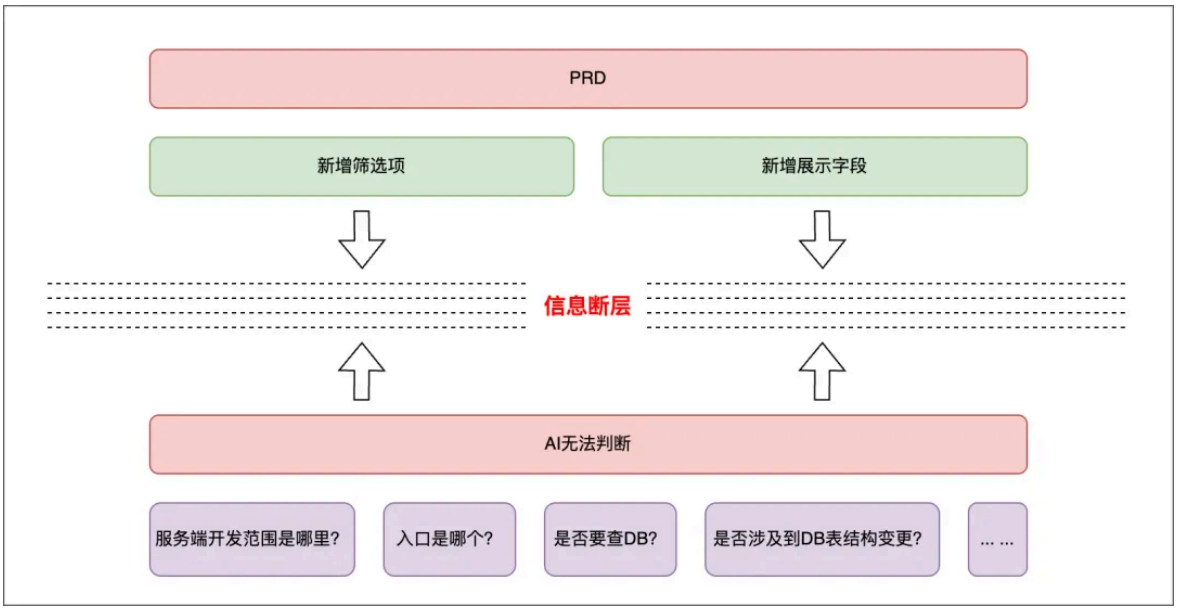
与前端“所见即所得”的开发模式截然不同，后端 AI Coding 面临的核心挑战在于PRD 并不等价于后端可执行的工程事实。实际上，后端的真实复杂度几乎全部隐藏在 PRD 之外，这些关键信息往往分散在成百上千个文件、模块与配置之中，PRD 既不可能也难以完整呈现。

2.1 PRD并不是后端可以直接执行的“工程语言”

从 PRD 出发，研发人员通常能够凭借经验自然地完成一系列“脑补”工作：明确哪些改动属于后端范畴、确定请求入口位置、判断是否涉及数据库操作、评估是否需要联动其他模块，以及查找系统中是否已存在相似逻辑等。

然而，这些对人类而言是再平常不过的常识，对 AI来说却如同信息断层。PRD 往往只会简单表述“前端/UI 需要改动，后端接口也要扩展字段”，却不会进一步说明：后端的开发边界究竟在哪里；改动是发生在 Controller 层、定时任务，还是 RPC 入口；字段是来源于 DB、下游服务，还是历史数据；改动是否会对缓存、索引或异步链路产生影响。

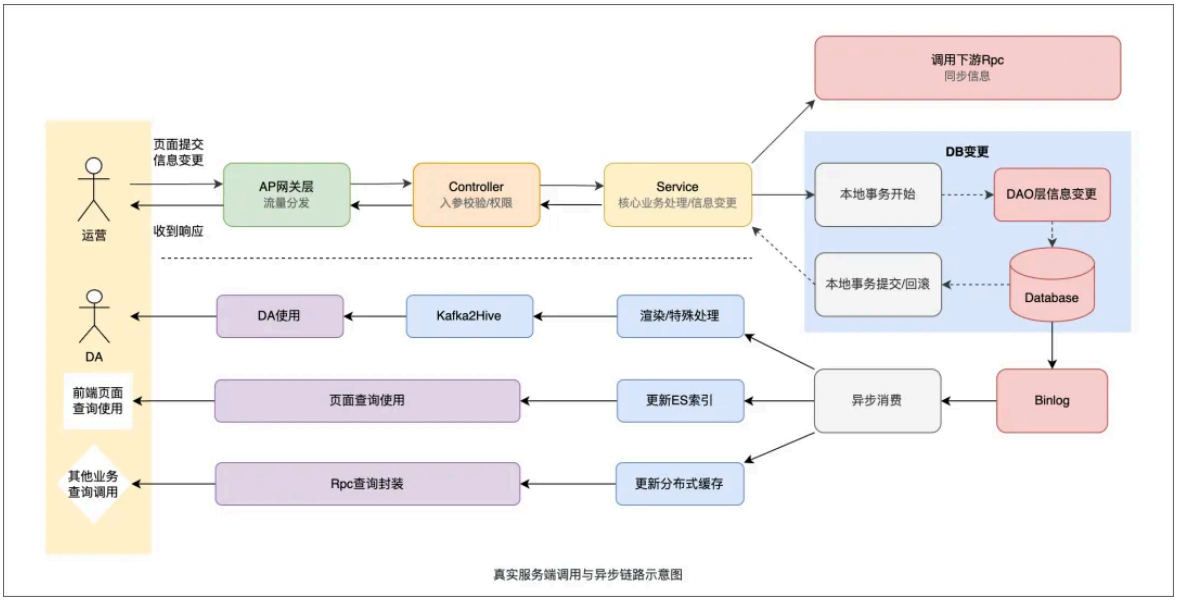
这些在日常开发中属于后端交付的最小必要信息，对 AI 而言，并非是“没看清楚”，而是压根就没有被提供。



2.2 后端真正的复杂度，几乎全部藏在 PRD 之外

更为关键的是，后端的复杂度并非体现在接口签名上，而是隐藏在流程和隐式约束之中。以一次“提交操作”为例，在真实的后端系统里，其背后往往涉及一系列复杂逻辑：（1）Service 内部的同步 RPC 调用；（2）本地事务控制与回滚逻辑；（3）数据库写入与 binlog 产生；（4）异步链路触发（如 Kafka、ES、缓存刷新）；（5）跨模块的数据一致性与最终状态收敛。

这些逻辑才是后端工程的“主体结构”，但 PRD 既不会也无法对这些流程进行详细描述。这就导致 AI 虽然能够看到“要改接口”，却无法预见一次改动会在系统中引发多少连锁反应。



三、从 Context 到 Control：后端 AI Coding 的两阶段演进

在探索 Java 后端 AI Coding 的过程中，我们并不是一开始就知道答案。恰恰相反，我们经历了两次明显的“卡住—反思—重构”的过程。回头来看，这两次演进背后，其实对应着两个逐渐清晰的认知：

- 第一阶段，致力于解决 AI 能否看到系统全貌（Context）的问题；
- 第二阶段，则聚焦于即便 AI 能看到系统全貌，能否实现稳定控制（Control）的挑战。

3.1 第一阶段：解决上下文问题

在前文的案例中，我们遇到的问题：AI 能修改局部代码，却无法理解它在系统中的位置。这并不是模型“不聪明”，而是因为它缺乏像人一样的系统视角。而这种视角，在后端工程中，来自于对三件事的理解：

1. 代码是如何分层的；
2. 各类代码元素之间是如何关联的；
3. 一次需求，会沿着怎样的调用与数据路径传播。

在前文提及的案例中，我们遭遇了这样的困境：AI能够修改局部代码，却无法理解这些代码在系统中的位置。这并非模型不够“聪明”，而是因为它缺乏像人类研发人员那样的系统视角。而在后端工程里，这种视角源于对三件事的理解：一是代码是如何分层的；二是各类代码元素之间是如何关联的；三是一次需求会沿着怎样的调用与数据路径传播。

3.1.1 明确代码层级结构

在研发人员的认知中，一个后端工程天然存在多种划分方式。从职责视角看，Controller、Service、DAO、Helper 各自承担着不同的职责；从系统边界视角看，又可划分为本服务、下游服务以及公共组件。

然而，这些划分往往仅存在于“共识”之中，并未被机器显式感知。因此，我们的第一步并非急于让 AI 编写代码，而是先将这些层级结构显性化，明确每一层的职责、哪些层之间允许直接交互，以及哪些改动天然具有“扩散效应”。以下是两种视角的代码层级结构示例：

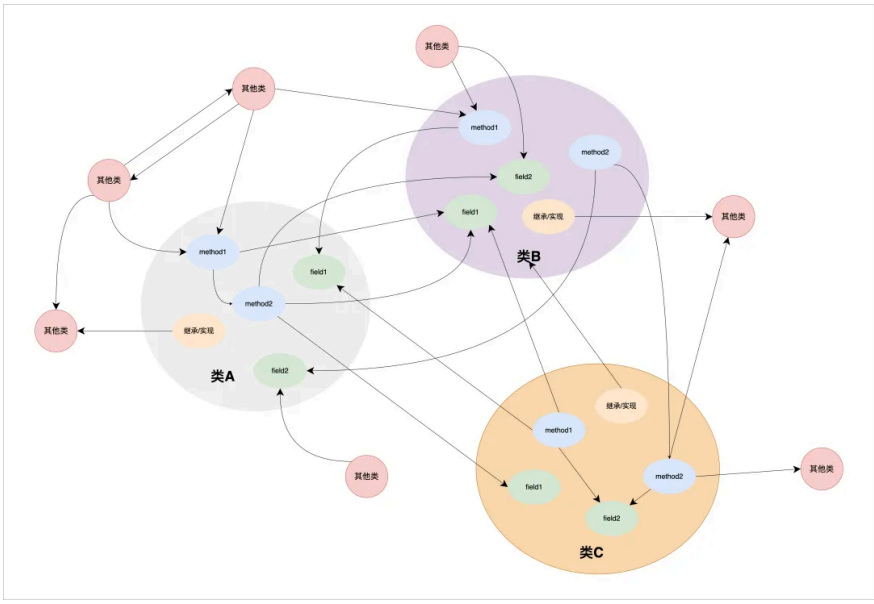
以下为两种视角的代码层级结构：

Controller	Service、Helper	DAO	VO	Util
处理用户请求	处理业务逻辑	操作数据库	数据传输	公共工具封装
按照系统内部功能职责				

Api	Component	Sdk
处理请求入口	核心业务逻辑模块化，负责实际业务处理	封装与外部系统交互的逻辑
按照系统边界和服务/组件间职责		

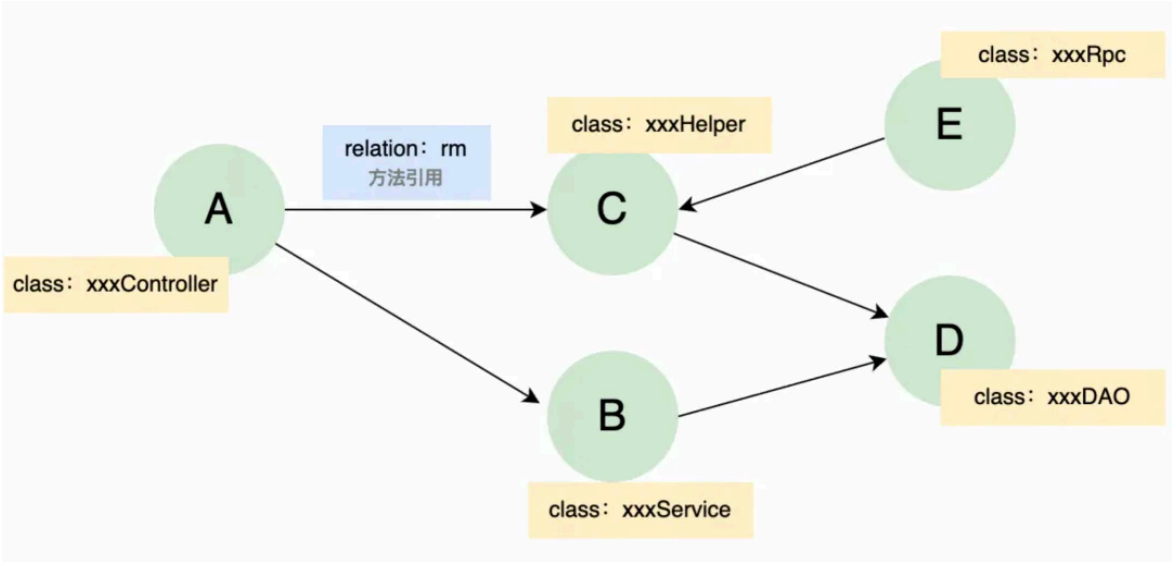
3.1.2 从“类文件”到“代码元素”

仅仅知晓“这是 Service、这是 DAO”还远远不够。在真实的工程环境中，改动并非发生在“类”这个粗粒度层面，而是发生在更细的代码元素之间，比如一个方法调用了谁、一个字段被哪些逻辑依赖、一个构造器是否隐式影响了初始化流程等。所以，我们进一步将分析粒度下沉至类中的字段、方法、构造器以及它们之间的关系，目的是让 AI 理解“修改的这一行代码，究竟会牵动哪些地方”。



3.1.3 构建“代码元素关系图谱”（Code Graph）

当我们将关注点从“文件”转向“代码元素及其关系”后，一种更自然的抽象方式出现了：整个代码库本质上是一张有向依赖图。其中，节点代表字段、方法、构造器等代码元素，边代表调用、引用、继承、依赖等关系。



数据结构定义：图的节点为类中元素，包括字段、方法、构造器等；
图的边为元素间的依赖关系，如类的继承、方法引用、字段引用等。

我们选择从编译后的字节码入手构建这张代码元素关系图谱，并将其存入图数据库中。这样做的好处在于，系统的“隐式结构”首次变成了可查询、可遍历、可计算的实体。

分析项目编译后的字节码文件，获取类中元素以及相互依赖关系，进而构建图数据库

3.1.4 从图谱到智能：AI 借助代码图谱获得系统视角

当代码关系以图的形式存在后，AI 不再需要“猜”系统结构。通过图数据库查询（如 Cypher），AI 可以明确知道：一个类的上下游依赖、一个方法被哪些路径调用、一次改动可能影响的范围。这一步，标志着 Context 不再只是静态文本，而是结构化、可推理的工程事实。

代码图谱存入 Neo4j 的图数据库中之后，可通过图数据库查询语言 Cypher，以直观的声明式语法实现高效的图遍历、路径分析等操作

最终，我们解决Context问题的设计分为三层：内容基座层、补充信息层以及大模型执行层。

在内容基座层，我们将整个代码库抽象为代码元素关系图谱，把工程事实转化为可计算的结构；在补充信息层，引入面向 AI 的 Context 约束与规范层，补齐 AI 做判断所需的关键信息，同时严格控制其发挥空间；在大模型执行层，大模型基于代码图谱生成 Cypher 查询，通过查询结果精确获取上下游依赖、影响范围与修改路径，在明确约束和规范的前提下做出工程级决策。

3.2 第二阶段：解决可控性问题

成功解决 Context 问题后，我们确实看到 AI Coding 的效果有了明显提升。但很快，新的问题出现了。

3.2.1 单Agent，无法稳定承载复杂任务

当我们尝试用一个大模型串联需求理解、方案推导、代码生成、多文件修改整个流程时，问题开始集中爆发，上下文迅速膨胀，Prompt 变得又长又脆弱、不同阶段的信息相互干扰、幻觉开始在“看似合理”的地方出现。这让我们意识到：Context解决的是“看不看得到”，但并没有解决“能不能被稳定使用”。

3.2.2 从 Prompt 驱动，到代码驱动的 Multi-Agent

真正的转折点在于我们放弃了“一个 Prompt 解决所有问题”的思路，转而引入 Multi-Agent / SubAgent 架构。该架构的核心目标有两个：一是控制单个 Agent 的上下文长度，避免信息过载；二是让不同阶段的任务彼此隔离，减少相互干扰。

在这种架构下，主控 Agent 负责任务拆解与结果汇总，子 Agent 只专注于单一子目标，代码修改本身交由 kwaipolit（快手内部自研的Copilot工具）完成，而非自由生成。如此一来，Workflow 从“Prompt 驱动”转向了代码驱动，确定性显著提升。

流程示意图

Multi-Agent执行流程示意

四、从“写代码”到“设计执行”：后端 AI Coding 的新范式

在前面的章节中，我们讨论了 Context 和 Control 如何让 AI 在后端工程中“看得见、走得稳”。接下来我们要描述的是当 AI 真的具备工程级执行能力之后，工程师的角色和工作内容会发生怎样的变化呢？

4.1 当AI成为工程执行者，工程师在做什么

在传统研发模式下，工程师往往同时扮演三种角色：需求理解者、方案设计者、具体实现者。而在后端 AI Coding 成熟之后，这三者开始出现清晰分离。在新的研发范式中，AI 不再是“补全代码的工具”，而是工程执行者；而工程师的核心职责，开始前移并上移。

在新的研发流程中，工程师不再直接编写最终代码，而是将精力集中在以下三个方面：

- 其一，从 PRD 中拆解模块边界：明确哪些改动属于同一个工程问题，哪些是不同子问题，为后续工作划定清晰的范围。
- 其二，运用流程图描述主干逻辑，通过 mermaid 等形式，把核心调用路径和数据流显性化，让复杂的系统逻辑一目了然。
- 其三，补齐关键参考类与入口信息，帮助系统快速定位工程中的“锚点”。

通过完成上述工作，工程师最终产出的是一份概要设计，而非实现代码。这份概要设计就如同建筑行业的施工蓝图，为后续的工程实施提供了清晰的指导方向。

在概要设计交付之后，后续工作不再依赖人工展开，AI从生成代码的角色转变成了执行设计。核心 Agent 读取概要设计、调度多个 SubAgent、分别完成图检索、代码检索、语义检索、进行场景理解与动作决策、生成一份可执行的编码计划。最终，在 IDE 中将编码计划转化为真实代码修改。

简而言之，在新的研发范式下，人类工程师负责“想清楚要做什么”，而 AI 则负责“在工程约束下把事情做完”，二者各司其职，共同推动项目的顺利进行。

在新的研发范式下，前文提及的落地失败的案例，概要设计如下：

4.2 整体架构

这套研发范式并不是“角色分工的口头约定”，而是由一整套工程架构托底的完整架构。

从系统视角看，整个码图体系被清晰地拆分为准备阶段与实施阶段。

- 在准备阶段，会对代码仓库做一次工程级建模。一方面，通过编译分析生成 Java 的调用依赖图谱并进行图结构化存储；另一方面借助大模型以及DeepWiki（快手内部结构化仓库说明文档）解析仓库结构、业务术语与工程规范，形成领域知识并向量化存储。最终，代码从“文件集合”被转化为“可检索、可推理的工程地图”。
- 在实施阶段，研发人员先基于 PRD 产出模块拆分和概要设计。码图的核心 Agent 读取这些设计后，通过本地 Python 驱动的可控执行引擎，不断调用图检索、代码检索与语义检索工具，进行场景理解与动作决策，最终生成一份伪代码级别的编码计划。

当编码计划生成后，系统将控制权交给 Kwaipilot，由其在工程约束下完成最终代码生成。

五、总结

有一句在计算机科学领域广为流传的话：“任何复杂问题，都可以通过引入一个中间层来解决。”在后端 AI Coding 这件事上，我们同样验证了这一点。在码图体系中，我们引入了两个关键的“中间层”：

- PRD → 概要设计：由人完成，用来保证需求理解的准确性与架构决策的质量
- 概要设计 → 编码计划：由独立 SubAgent 完成，用来将设计翻译为可执行、可校验的工程步骤

正是这两个中间层，把“需求”与“代码”之间那段最容易失控的黑盒过程，拆解成了可理解、可验证、可约束的工程路径。AI 也因此从“猜需求写代码”，转变为“按工程计划执行代码”。

经过多轮方案演进和真实生产环境的验证，我们逐渐形成了两个非常清晰、也非常重要的共识。

第一，精准的 Context 是后端 AI Coding 的决定性因素。倘若没有上下文，AI 只能在局部文件中补全和改写，难以把握系统的全貌。只有当代码被图谱化、被语义化、被工程化之后，AI 才能真正理解系统级结构，承担真实需求的实现工作。

第二，提升可控性，是所有 “AI Coding 产品”必须优先解决的核心能力。可控性并不是通过更强的模型能力自然获得的，而是由模型能力、Prompt 设计、系统框架与工程校验共同构成的一种体系化能力。当 Context 被工程化，当行为被系统约束，AI 才第一次具备了成为“工程执行者”的资格。

【END】

【相关阅读】